A Complete Hands-On Workbook

Learn Kubernetes By Doing

30	10	300+
Hands-On Assignments	Sections & Topics	kubectl Commands

WHAT YOU WILL LEARN

- Pods, Deployments & ReplicaSets
- Services — ClusterIP, NodePort, LB
- Ingress routing & TLS termination
- Rolling updates & rollbacks
- ConfigMaps & Secrets injection
- PersistentVolumes & PVCs
- StatefulSets for databases
- RBAC, Roles & ServiceAccounts
- HPA & cluster autoscaling
- NetworkPolicies & zero trust
- Jobs, CronJobs & DaemonSets
- Helm — install, build, upgrade
- Init containers & sidecars
- Security contexts & hardening
- kubectl mastery & JSONPath
- PodDisruptionBudgets for HA

BEGINNER
Core concepts & first commands

INTERMEDIATE
Multi-resource workflows

ADVANCED
Production-grade patterns

TOOLS & PLATFORMS COVERED

kubectl · Helm · Minikube · kind · EKS/AKS/GKE · GitHub Actions · Prometheus · Grafana · Trivy

Aditya Jaiswal
YouTube • @devopsshack • LinkedIn • Instagram

How to Use This Book

This workbook is designed on a simple principle: the fastest way to learn Kubernetes is to run commands, break things, fix them, and understand why they work. Each assignment follows the same structure:

- Objective: What you will accomplish
- Prerequisites: What you need before starting
- Task: The actual hands-on problem to solve
- Solution: Step-by-step commands with YAML manifests
- Explanation: The 'why' behind every concept and command
- Key Takeaways: What to remember for real-world use

Prerequisites for All Assignments

A running Kubernetes cluster (Minikube, kind, k3s, or a cloud cluster). kubectl installed and configured. Basic Linux command-line familiarity. Docker installed for container-related assignments.

Difficulty	What It Means
`Beginner`	Core concepts, basic commands, first-time setups
`Intermediate`	Multi-resource workflows, debugging, configuration
`Advanced`	Production patterns, RBAC, networking, scaling

SECTION 1 Core Fundamentals — Pods, Deployments & Services

Assignment 01 **Your First Pod — Run an Nginx Container**	Difficulty **Beginner**	Est. Time **15 min**

Objective

Create and inspect your first Kubernetes Pod running an Nginx web server. Understand what a Pod is and how to interact with it.

Task

- Create a Pod named my-nginx running the nginx:latest image
- Verify the Pod is running
- Get the Pod's IP address
- View the logs from the container
- Execute a shell command inside the Pod
- Delete the Pod

Solution

```
# Step 1 - Create the Pod imperatively
kubectl run my-nginx --image=nginx:latest --port=80

# Step 2 - Watch the Pod come up
kubectl get pods -w

# Step 3 - Describe the Pod (IP, events, node)
kubectl describe pod my-nginx

# Step 4 - Get Pod IP only
kubectl get pod my-nginx -o jsonpath='{.status.podIP}'

# Step 5 - View container logs
kubectl logs my-nginx

# Step 6 - Execute shell inside the Pod
kubectl exec -it my-nginx -- /bin/bash
# Inside the shell:
curl localhost
exit

# Step 7 - Generate the YAML definition (for learning)
kubectl get pod my-nginx -o yaml

# Step 8 - Delete the Pod
kubectl delete pod my-nginx
```

Explanation

Pod: A Pod is the smallest deployable unit in Kubernetes. It wraps one or more containers that share network (same IP) and storage. Containers inside the same Pod communicate via localhost.

kubectl run: The imperative command to quickly spin up a Pod. In production you would use declarative YAML, but imperative commands are perfect for quick tests.

kubectl describe: Your best friend for debugging. It shows the full lifecycle of a resource including Events at the bottom which tell you exactly why something succeeded or failed.

kubectl exec -it: Opens an interactive terminal inside a running container, similar to 'docker exec'. The -i flag keeps stdin open, -t allocates a TTY.

> **Key Takeaway**
> Pods are ephemeral — when deleted, they are gone. You never manage Pods directly in production. You create Deployments which manage Pods for you.

Assignment 02 **Declarative Pods — Writing Your First YAML**	Difficulty **Beginner**	Est. Time **20 min**

Objective

Create a Pod using a YAML manifest instead of imperative commands. Learn the four required fields of every Kubernetes object.

Task

- Write a YAML manifest for a Pod running busybox that sleeps for 3600 seconds
- Apply it with kubectl apply
- Update the sleep duration to 7200 and apply again
- Understand dry-run and diff

Solution

```yaml
# pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: busybox-pod
  labels:
    app: busybox
    env: learning
spec:
  containers:
  - name: busybox
    image: busybox:1.35
    command: ['sh', '-c', 'sleep 3600']
    resources:
      requests:
        cpu: 50m
        memory: 64Mi
      limits:
        cpu: 100m
```

```
        memory: 128Mi

# Apply the manifest
kubectl apply -f pod.yaml

# Dry-run to validate before applying
kubectl apply -f pod.yaml --dry-run=client

# Server-side dry run (more accurate)
kubectl apply -f pod.yaml --dry-run=server

# See diff before applying changes
kubectl diff -f pod.yaml

# Update sleep to 7200 in pod.yaml, then:
kubectl apply -f pod.yaml
```

Explanation

Four Required Fields: Every Kubernetes object needs apiVersion (which API to use), kind (what type of object), metadata (name, namespace, labels), and spec (desired state of the object).

Labels: Key-value pairs attached to objects. They are used for selecting, filtering, and grouping resources. Labels are how Services find Pods and how Deployments track their Pods.

Resource Requests vs Limits: Requests are what the scheduler uses to decide which node to place the Pod on. Limits are the maximum the container can consume. Always set both in production.

kubectl apply vs kubectl create: apply is idempotent and declarative — it creates OR updates. create fails if the resource already exists. Always use apply with YAML files.

Assignment 03 **Deployments — Self-Healing and Scaling**	Difficulty **Beginner**	Est. Time **25 min**

Objective

Create a Deployment, scale it, simulate a Pod crash, and watch Kubernetes self-heal. Understand ReplicaSets.

Task

- Create a Deployment named web-app running nginx with 3 replicas
- Scale it to 5 replicas imperatively
- Delete one Pod and watch it get recreated
- View the rollout history

Solution

```
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
```

```
  name: web-app
  labels:
    app: web-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web-app
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app: web-app
    spec:
      containers:
      - name: nginx
        image: nginx:1.25
        ports:
        - containerPort: 80
        resources:
          requests:
            cpu: 100m
            memory: 128Mi
          limits:
            cpu: 200m
            memory: 256Mi

kubectl apply -f deployment.yaml
kubectl get deployment web-app
kubectl get pods -l app=web-app

# Scale to 5
kubectl scale deployment web-app --replicas=5
kubectl get pods -l app=web-app -w

# Delete a Pod — watch it come back
kubectl delete pod <any-pod-name>
kubectl get pods -w   # new Pod appears

# View rollout history
kubectl rollout history deployment web-app
```

Explanation

ReplicaSet: A Deployment creates a ReplicaSet which ensures the specified number of Pod replicas are running at all times. If a Pod dies, the ReplicaSet controller detects the drift and creates a new one.

Selector & matchLabels: This is how the Deployment knows which Pods belong to it. The selector must match the template's labels. This is a very common source of bugs — a mismatch means zero Pods get managed.

RollingUpdate Strategy: maxSurge=1 means Kubernetes can create 1 extra Pod above the desired count during an update. maxUnavailable=0 means no Pod should be unavailable. Combined: zero-downtime deploys.

> **Key Takeaway**
> Never create standalone Pods for applications. Always use Deployments. Deployments give you self-healing, scaling, rolling updates, and rollback — all for free.

Assignment 04 **Services — Expose Your Application**	Difficulty **Beginner**	Est. Time **25 min**

Objective

Expose a Deployment using ClusterIP, NodePort, and LoadBalancer Service types. Understand how Service discovery works.

Task

- Create a Deployment with 3 nginx replicas
- Expose it as a ClusterIP Service
- Access it from inside the cluster using a temporary Pod
- Expose it as a NodePort Service
- Understand the difference between all Service types

Solution

```
# clusterip-svc.yaml
apiVersion: v1
kind: Service
metadata:
  name: web-app-clusterip
spec:
  selector:
    app: web-app
  ports:
  - port: 80
    targetPort: 80
    protocol: TCP
  type: ClusterIP

kubectl apply -f clusterip-svc.yaml
kubectl get svc web-app-clusterip

# Test from inside cluster
kubectl run test-pod --image=curlimages/curl -it --rm \
  -- curl web-app-clusterip

# NodePort Service
```

```
apiVersion: v1
kind: Service
metadata:
  name: web-app-nodeport
spec:
  selector:
    app: web-app
  ports:
  - port: 80
    targetPort: 80
    nodePort: 30080
  type: NodePort

kubectl apply -f nodeport-svc.yaml
# Access: http://<node-ip>:30080
minikube service web-app-nodeport --url   # for Minikube
```

Explanation

ClusterIP (default): Creates a stable virtual IP reachable only inside the cluster. All intra-cluster communication uses ClusterIP Services. The Service name becomes a DNS entry automatically.

NodePort: Opens a port (30000-32767) on every node in the cluster. Traffic to any-node-ip:nodePort gets routed to the Service. Good for development, not recommended for production.

LoadBalancer: Provisions an external cloud load balancer. Only works on cloud providers (AWS, GCP, Azure). On Minikube, use 'minikube tunnel' to simulate this.

Service Discovery: kube-dns creates a DNS entry for every Service: <service-name>.<namespace>.svc.cluster.local. Pods in the same namespace can use just the service name.

Service Type	Accessibility
`ClusterIP`	Internal cluster only (default)
`NodePort`	External via node IP and high port
`LoadBalancer`	External via cloud load balancer IP
`ExternalName`	Maps to an external DNS name

Assignment 05 **Rolling Updates and Rollbacks**	Difficulty **Beginner**	Est. Time **30 min**

Objective

Perform a zero-downtime rolling update of a Deployment and practice rolling back to a previous version.

Task

- Deploy nginx:1.24 with 4 replicas
- Update the image to nginx:1.25 and observe the rolling update
- Simulate a bad deploy using nginx:broken-image
- Rollback to the previous working version

Solution

```
# Deploy initial version
kubectl create deployment web-app --image=nginx:1.24 --replicas=4
kubectl rollout status deployment web-app

# Update image (rolling update begins)
kubectl set image deployment/web-app nginx=nginx:1.25
kubectl rollout status deployment web-app

# Check rollout history
kubectl rollout history deployment web-app

# Annotate the change for clarity
kubectl annotate deployment web-app \
  kubernetes.io/change-cause='Update to nginx 1.25'

# Simulate a bad image
kubectl set image deployment/web-app nginx=nginx:this-does-not-exist
kubectl rollout status deployment web-app  # will show waiting
kubectl get pods   # ImagePullBackOff on new pods

# Rollback to previous revision
kubectl rollout undo deployment web-app
kubectl rollout status deployment web-app

# Rollback to a specific revision
kubectl rollout undo deployment web-app --to-revision=1
```

Explanation

Rolling Update: Kubernetes replaces old Pods with new ones gradually. It creates a new ReplicaSet with the new image, scales it up, then scales down the old one. This ensures no downtime.

Rollback: Kubernetes stores rollout history. 'rollout undo' switches back to the previous ReplicaSet. The old ReplicaSet was kept with 0 replicas, so rollback is instant.

ImagePullBackOff: The container runtime could not pull the image. Usually means wrong image name, tag, or missing registry credentials. A Pod stuck in ImagePullBackOff does not get killed — Kubernetes keeps trying.

SECTION 2 Configuration — ConfigMaps, Secrets & Environment

Assignment 06

ConfigMaps — Externalize Configuration

Difficulty	Est. Time
Beginner	25 min

Objective

Create ConfigMaps and inject them into Pods as environment variables and as mounted volume files.

Task

- Create a ConfigMap with application settings
- Inject ConfigMap values as env variables into a Pod
- Mount the ConfigMap as a file inside the Pod
- Update the ConfigMap and observe the change

Solution

```
# configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_ENV: production
  APP_PORT: '8080'
  LOG_LEVEL: info
  database.conf: |
    host=postgres.default.svc.cluster.local
    port=5432
    name=myapp
    pool_size=10

kubectl apply -f configmap.yaml
kubectl describe configmap app-config

# pod-with-config.yaml
apiVersion: v1
kind: Pod
metadata:
  name: config-demo
spec:
  containers:
  - name: app
    image: busybox
    command: ['sh', '-c', 'env && cat /config/database.conf && sleep 3600']
    env:
    - name: APP_ENV
      valueFrom:
        configMapKeyRef:
```

```
          name: app-config
          key: APP_ENV
    - name: APP_PORT
      valueFrom:
        configMapKeyRef:
          name: app-config
          key: APP_PORT
    envFrom:
    - configMapRef:
        name: app-config
    volumeMounts:
    - name: config-volume
      mountPath: /config
  volumes:
  - name: config-volume
    configMap:
      name: app-config
      items:
      - key: database.conf
        path: database.conf
```

Explanation

ConfigMap: Stores non-sensitive configuration as key-value pairs. Decouples config from container images so you can change config without rebuilding images.

env vs envFrom: env lets you pick specific keys. envFrom loads ALL keys from the ConfigMap as environment variables. Use env when you need to rename keys.

Volume Mount: Mounts ConfigMap data as files. This is useful for config files (nginx.conf, app.properties). When you update the ConfigMap, the files update automatically (within ~1 minute) without restarting the Pod.

> **Key Takeaway**
> Env variables are baked into the Pod at startup and do NOT update when ConfigMap changes. Volume-mounted files DO update dynamically. Choose based on whether your app supports live config reload.

Assignment 07 **Secrets — Storing Sensitive Data**	Difficulty **Intermediate**	Est. Time **30 min**

Objective

Create Kubernetes Secrets for database credentials and TLS certificates. Inject them securely into Pods.

Task

- Create a Secret for database credentials imperatively
- Create a TLS Secret from certificate files
- Inject Secret values as environment variables
- Mount a Secret as a volume for TLS certs

- Decode a Secret value using kubectl

Solution

```
# Create generic secret imperatively
kubectl create secret generic db-credentials \
  --from-literal=DB_USER=admin \
  --from-literal=DB_PASSWORD=SuperSecret123 \
  --from-literal=DB_HOST=postgres.default.svc.cluster.local

# View the secret (values are base64 encoded)
kubectl get secret db-credentials -o yaml

# Decode a value
kubectl get secret db-credentials -o jsonpath='{.data.DB_PASSWORD}' | base64 -d

# Declarative secret (base64 encode values first)
echo -n 'admin' | base64       # YWRtaW4=
echo -n 'SuperSecret123' | base64

# secret.yaml
apiVersion: v1
kind: Secret
metadata:
  name: db-credentials
type: Opaque
data:
  DB_USER: YWRtaW4=
  DB_PASSWORD: U3VwZXJTZWNyZXQxMjM=

# Pod using the secret
spec:
  containers:
  - name: app
    image: busybox
    command: ['sh','-c','echo $DB_USER && sleep 3600']
    env:
    - name: DB_USER
      valueFrom:
        secretKeyRef:
          name: db-credentials
          key: DB_USER
    - name: DB_PASSWORD
      valueFrom:
        secretKeyRef:
          name: db-credentials
          key: DB_PASSWORD
```

Explanation

Base64 Encoding: Secrets store data as base64, NOT encryption. Base64 is encoding for safe transport of binary data, not a security measure. Anyone with cluster access can decode it easily.

Secret Types: Opaque is the generic type. kubernetes.io/tls is for TLS certs. kubernetes.io/dockerconfigjson is for private registry credentials. Using the right type enables validation.

Production Secret Management: In production, use external secret managers like HashiCorp Vault, AWS Secrets Manager, or the External Secrets Operator. Never commit plain Secret YAML with values to Git — use sealed-secrets or SOPS.

Security Warning

kubectl get secret shows base64 values to anyone with read access. Enable RBAC, enable etcd encryption at rest, and consider using External Secrets Operator for production workloads.

Assignment 08 **Resource Management — Requests, Limits & LimitRanges**	Difficulty **Intermediate**	Est. Time **30 min**

Objective

Configure resource requests and limits for containers. Create a LimitRange to enforce defaults across a namespace.

Task

- Create a namespace called resource-lab
- Apply a LimitRange that sets default CPU/memory
- Create Pods with and without explicit resource requests
- Simulate an OOMKilled container
- Create a ResourceQuota for the namespace

Solution

```
# Create namespace
kubectl create namespace resource-lab

# limitrange.yaml
apiVersion: v1
kind: LimitRange
metadata:
  name: default-limits
  namespace: resource-lab
spec:
  limits:
  - type: Container
    default:
      cpu: 200m
      memory: 256Mi
    defaultRequest:
      cpu: 100m
      memory: 128Mi
    max:
```

```
      cpu: '2'
      memory: 1Gi
    min:
      cpu: 50m
      memory: 64Mi

kubectl apply -f limitrange.yaml

# resourcequota.yaml
apiVersion: v1
kind: ResourceQuota
metadata:
  name: namespace-quota
  namespace: resource-lab
spec:
  hard:
    pods: '10'
    requests.cpu: '4'
    requests.memory: 4Gi
    limits.cpu: '8'
    limits.memory: 8Gi

kubectl apply -f resourcequota.yaml
kubectl describe resourcequota -n resource-lab

# Simulate OOMKilled - set very low memory limit
kubectl run oom-demo --image=polinux/stress \
  --limits='memory=64Mi' \
  --command -- stress --vm 1 --vm-bytes 128M
kubectl get pod oom-demo -w   # STATUS: OOMKilled
```

Explanation

Requests: Used by the scheduler. A node must have at least this much free capacity to schedule the Pod. Think of it as a reservation.

Limits: Enforced at runtime by the kubelet. CPU is throttled (not killed) when exceeded. Memory is killed when exceeded — this is OOMKilled (Out Of Memory Killed).

LimitRange: Applies defaults to containers that don't specify requests/limits. Without LimitRanges, a container with no limits can consume all node resources.

ResourceQuota: Caps total resource consumption for an entire namespace. Prevents one team from starving others in a multi-tenant cluster.

SECTION 3 Storage — Volumes, PVs, PVCs & StatefulSets

Assignment 09

Persistent Volumes & Claims

Difficulty	Est. Time
Intermediate	35 min

Objective

Provision a PersistentVolume, claim it with a PVC, mount it in a Pod, write data, delete the Pod, and verify data persists.

Task

- Create a 1Gi hostPath PersistentVolume
- Create a PVC requesting 500Mi of storage
- Mount the PVC into a Pod and write a file
- Delete the Pod
- Create a new Pod and verify the file still exists

Solution

```
# pv.yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: local-pv
spec:
  capacity:
    storage: 1Gi
  accessModes:
  - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  storageClassName: manual
  hostPath:
    path: /data/k8s-pv
    type: DirectoryOrCreate

# pvc.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-pvc
spec:
  accessModes:
  - ReadWriteOnce
  storageClassName: manual
  resources:
    requests:
      storage: 500Mi

kubectl apply -f pv.yaml
```

```
kubectl apply -f pvc.yaml
kubectl get pv,pvc   # STATUS should be Bound

# Pod writing to PVC
spec:
  containers:
  - name: writer
    image: busybox
    command: ['sh','-c','echo DevOps Shack > /data/hello.txt && sleep 3600']
    volumeMounts:
    - name: persistent-storage
      mountPath: /data
  volumes:
  - name: persistent-storage
    persistentVolumeClaim:
      claimName: my-pvc

# Verify data then delete pod and re-create another pod
kubectl exec writer-pod -- cat /data/hello.txt
kubectl delete pod writer-pod
# New pod mounts same PVC — data is there!
```

Explanation

PersistentVolume (PV): A piece of storage provisioned by an admin (or dynamically). It exists independently of any Pod. hostPath uses the node's filesystem — only suitable for single-node clusters like Minikube.

PersistentVolumeClaim (PVC): A request for storage by a user. Kubernetes matches PVCs to suitable PVs based on access modes, storage class, and size. Once bound, the PVC exclusively owns that PV.

Access Modes: ReadWriteOnce (one node read-write), ReadOnlyMany (many nodes read-only), ReadWriteMany (many nodes read-write). Most cloud block volumes only support RWO.

Reclaim Policy: Retain keeps the PV and its data after the PVC is deleted (admin must clean up manually). Delete removes the PV and its data. Recycle is deprecated.

Assignment 10 **StatefulSets — Databases and Ordered Workloads**	Difficulty **Intermediate**	Est. Time **40 min**

Objective

Deploy a stateful application (Redis) using a StatefulSet. Understand ordered Pod creation, stable hostnames, and per-Pod PVCs.

Task

- Create a StatefulSet for Redis with 3 replicas
- Create a Headless Service for stable DNS
- Observe ordered Pod creation (redis-0, redis-1, redis-2)
- Delete redis-1 and watch it come back with the same name and storage

Solution

```
# headless-svc.yaml
apiVersion: v1
kind: Service
metadata:
  name: redis-headless
spec:
  clusterIP: None   # This makes it Headless
  selector:
    app: redis
  ports:
  - port: 6379

# statefulset.yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: redis
spec:
  serviceName: redis-headless
  replicas: 3
  selector:
    matchLabels:
      app: redis
  template:
    metadata:
      labels:
        app: redis
    spec:
      containers:
      - name: redis
        image: redis:7.2-alpine
        ports:
        - containerPort: 6379
        volumeMounts:
        - name: redis-data
          mountPath: /data
  volumeClaimTemplates:
  - metadata:
      name: redis-data
    spec:
      accessModes: ['ReadWriteOnce']
      resources:
        requests:
          storage: 1Gi

kubectl apply -f headless-svc.yaml
kubectl apply -f statefulset.yaml
kubectl get pods -w   # redis-0, redis-1, redis-2 in order

# DNS of each Pod
```

```
# redis-0.redis-headless.default.svc.cluster.local
# redis-1.redis-headless.default.svc.cluster.local

# Delete redis-1 - it comes back as redis-1 with same PVC
kubectl delete pod redis-1
kubectl get pods -w
```

Explanation

StatefulSet vs Deployment: Deployments treat Pods as interchangeable. StatefulSets give each Pod a stable identity: a stable name (redis-0), a stable network hostname, and its own dedicated PVC.

Headless Service: clusterIP: None creates a DNS entry for EACH Pod individually (not a load-balanced VIP). This lets clients discover and connect to specific Pods — essential for databases.

volumeClaimTemplates: StatefulSets create a dedicated PVC for each Pod automatically. When redis-1 comes back, it reattaches to its original redis-data-redis-1 PVC.

Ordered Creation/Deletion: Pods are created 0, 1, 2... in order and deleted 2, 1, 0 in reverse. Each Pod must be Running before the next is created. This ensures leader election and initialization work correctly.

SECTION 4 Networking — Ingress, Network Policies & DNS

Assignment 11

Ingress — Route HTTP Traffic to Multiple Services

Difficulty	Est. Time
Intermediate	40 min

Objective

Deploy an Ingress controller and create Ingress rules to route traffic to multiple backend services based on path and host.

Task

- Install the Nginx Ingress controller
- Deploy two applications: frontend and api
- Create an Ingress routing /api/* to the api service and /* to frontend
- Add host-based routing for app.example.com

Solution

```
# Install Nginx Ingress (Minikube)
minikube addons enable ingress

# Install on any cluster
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/\
  controller-v1.10.0/deploy/static/provider/cloud/deploy.yaml

kubectl get pods -n ingress-nginx -w

# Deploy frontend
kubectl create deployment frontend --image=nginx
kubectl expose deployment frontend --port=80 --name=frontend-svc

# Deploy api (using a different page)
kubectl create deployment api --image=hashicorp/http-echo \
  --env='ECHO_TEXT=Hello from API'
kubectl expose deployment api --port=5678 --name=api-svc

# ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /$2
spec:
  ingressClassName: nginx
  rules:
  - host: app.example.com
    http:
      paths:
```

```
      - path: /api(/|$)(.*)
        pathType: Prefix
        backend:
          service:
            name: api-svc
            port:
              number: 5678
      - path: /
        pathType: Prefix
        backend:
          service:
            name: frontend-svc
            port:
              number: 80

kubectl apply -f ingress.yaml
kubectl get ingress

# Add to /etc/hosts for local testing
echo '127.0.0.1 app.example.com' >> /etc/hosts
curl http://app.example.com/
curl http://app.example.com/api/
```

Explanation

Ingress: An API object that manages external HTTP/HTTPS access to Services inside the cluster. It provides URL-based routing, TLS termination, and virtual hosting — all in one place.

Ingress Controller: The Ingress resource is just config. You need an Ingress Controller (a running process) to act on it. The Nginx Ingress Controller is the most popular. Others include Traefik, HAProxy, and cloud-native ones.

PathType: Prefix matches the path and any subpath. Exact requires a perfect match. ImplementationSpecific delegates to the controller.

Annotations: Ingress controllers add extra power via annotations. nginx.ingress.kubernetes.io/rewrite-target rewrites the URL path before sending to the backend, useful when your service doesn't have the /api prefix.

Assignment 12 **Network Policies — Isolate Your Workloads**	Difficulty **Advanced**	Est. Time **40 min**

Objective

Implement NetworkPolicies to restrict traffic between Pods. Start with a default-deny policy, then selectively allow traffic.

Task

- Deploy frontend, backend, and database Pods in separate label groups
- Apply default-deny ingress to the database namespace
- Allow only the backend to connect to the database

- Allow only the frontend to connect to the backend
- Verify isolation is working using curl from different Pods

Solution

```
# Setup three deployments
kubectl run frontend --image=nginx --labels='tier=frontend'
kubectl run backend --image=nginx --labels='tier=backend'
kubectl run database --image=nginx --labels='tier=database'

# default-deny.yaml - deny all ingress to database
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-ingress
spec:
  podSelector:
    matchLabels:
      tier: database
  policyTypes:
  - Ingress

# allow-backend-to-db.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-backend-to-db
spec:
  podSelector:
    matchLabels:
      tier: database
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          tier: backend
    ports:
    - protocol: TCP
      port: 80

kubectl apply -f default-deny.yaml
kubectl apply -f allow-backend-to-db.yaml

# Test: frontend cannot reach database
kubectl exec frontend -- curl --max-time 3 database  # times out

# Test: backend CAN reach database
kubectl exec backend -- curl --max-time 3 database   # succeeds
```

Explanation

NetworkPolicy: Kubernetes firewall rules for Pods. Without any NetworkPolicy, all Pods can talk to all other Pods. Once you apply a policy to a Pod, only explicitly allowed traffic is permitted.

Default Deny Pattern: Best practice is to start with a default-deny policy, then add allow rules. This is the zero-trust networking model — deny everything, permit only what is needed.

podSelector: Selects which Pods the policy applies to (the target/destination). The from/to sections use podSelector and namespaceSelector to define allowed sources/destinations.

CNI Requirement: NetworkPolicies are only enforced if your cluster uses a CNI plugin that supports them: Calico, Cilium, Weave Net, or Antrea. Flannel does NOT enforce NetworkPolicies.

SECTION 5 Scheduling, Health Checks & Reliability

Assignment 13 **Health Checks — Liveness, Readiness & Startup Probes**	Difficulty **Intermediate**	Est. Time **35 min**

Objective

Configure all three probe types to make your application production-grade. Simulate failures and watch Kubernetes respond.

Task

- Create a Pod with an HTTP liveness probe
- Create a Pod with a readiness probe that delays traffic
- Create a slow-starting app with a startup probe
- Simulate a liveness failure and watch the container restart

Solution

```yaml
# probes-demo.yaml
apiVersion: v1
kind: Pod
metadata:
  name: probes-demo
spec:
  containers:
  - name: app
    image: nginx
    ports:
    - containerPort: 80
    startupProbe:
      httpGet:
        path: /
        port: 80
      failureThreshold: 30
      periodSeconds: 10
      # Allows up to 300s for slow startup
    livenessProbe:
      httpGet:
        path: /healthz
        port: 80
      initialDelaySeconds: 5
      periodSeconds: 10
      failureThreshold: 3
      timeoutSeconds: 2
    readinessProbe:
      httpGet:
        path: /ready
        port: 80
```

```
      initialDelaySeconds: 5
      periodSeconds: 5
      successThreshold: 1
      failureThreshold: 3

# Simulate liveness failure
kubectl exec probes-demo -- rm /usr/share/nginx/html/healthz
kubectl describe pod probes-demo   # see restart count go up
kubectl get pod probes-demo -w     # RESTARTS column increases
```

Explanation

Liveness Probe: Answers: Is this container alive? If it fails failureThreshold times, kubelet kills and restarts the container. Use for detecting deadlocks or hung processes.

Readiness Probe: Answers: Is this container ready to receive traffic? If it fails, the Pod is removed from the Service's Endpoints. Traffic stops going to it but the container is NOT restarted. Use for slow dependency connections.

Startup Probe: Answers: Has this container finished initializing? It disables liveness/readiness until it succeeds. Use for slow-starting apps to avoid premature liveness kills.

Probe Type	Action on Failure
`Liveness`	Container is killed and restarted
`Readiness`	Pod removed from Service Endpoints (no traffic)
`Startup`	Container is killed (liveness/readiness are paused)

Assignment 14

Node Affinity & Taints/Tolerations

Difficulty **Advanced** | Est. Time **40 min**

Objective

Control Pod scheduling using node labels, affinity rules, taints, and tolerations. Understand how to dedicate nodes for specific workloads.

Task

- Label a node with disktype=ssd and zone=us-east-1a
- Create a Pod with required node affinity for ssd disks
- Taint a node with gpu=true:NoSchedule
- Create a Pod with toleration to schedule on the tainted node
- Verify Pods land on the correct nodes

Solution

```
# Label nodes
kubectl label node <node-name> disktype=ssd zone=us-east-1a
kubectl get nodes --show-labels

# node-affinity.yaml
apiVersion: v1
```

```
kind: Pod
metadata:
  name: ssd-pod
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
        - matchExpressions:
          - key: disktype
            operator: In
            values:
            - ssd
      preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 80
        preference:
          matchExpressions:
          - key: zone
            operator: In
            values:
            - us-east-1a
  containers:
  - name: app
    image: nginx

# Taint a node
kubectl taint node <node-name> gpu=true:NoSchedule

# Pod that tolerates the taint
spec:
  tolerations:
  - key: gpu
    operator: Equal
    value: 'true'
    effect: NoSchedule
  containers:
  - name: ml-workload
    image: tensorflow/tensorflow:latest-gpu
```

Explanation

Node Affinity: Required rules (requiredDuringScheduling) prevent scheduling if not matched — the Pod stays Pending. Preferred rules (preferredDuringScheduling) are soft — the scheduler tries to honor them but will schedule elsewhere if needed.

Taints: Marks a node to repel Pods. NoSchedule means new Pods without tolerations won't be scheduled. NoExecute also evicts existing Pods without tolerations. PreferNoSchedule is a soft repel.

Tolerations: Allow a Pod to be scheduled onto tainted nodes. A toleration does not force placement — it just allows it. Combine with affinity to both allow AND require a specific node.

Real-World Use: Use taints to dedicate GPU nodes, spot/preemptible instances, or high-memory nodes to specific workloads. Control planes are tainted NoSchedule by default so user Pods don't land there.

Assignment 15 **Horizontal Pod Autoscaler (HPA)**	Difficulty **Advanced**	Est. Time **40 min**

Objective

Configure HPA to automatically scale a Deployment based on CPU utilization. Generate load and watch Kubernetes scale out then back in.

Task

- Deploy an app with resource requests set
- Install Metrics Server
- Create an HPA targeting 50% CPU utilization
- Generate load using a busybox loop
- Watch the HPA scale out then scale in when load stops

Solution

```
# Install Metrics Server (Minikube)
minikube addons enable metrics-server

# Install on any cluster
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/\
  releases/latest/download/components.yaml

# Deploy CPU-intensive app
kubectl create deployment php-apache \
  --image=registry.k8s.io/hpa-example \
  --requests='cpu=200m'
kubectl expose deployment php-apache --port=80

# Create HPA
kubectl autoscale deployment php-apache \
  --cpu-percent=50 \
  --min=1 \
  --max=10

# Or via YAML
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: php-apache
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
```

```
    name: php-apache
  minReplicas: 1
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 50

# Generate load in another terminal
kubectl run load-generator --image=busybox \
  -- /bin/sh -c 'while true; do wget -q -O- php-apache; done'

# Watch the HPA and pods scale
kubectl get hpa php-apache -w
kubectl get pods -w

# Stop load
kubectl delete pod load-generator
# Watch HPA scale back down (takes ~5 minutes default cooldown)
```

Explanation

HPA: Watches metrics and adjusts the replica count to maintain the target metric value. It uses the formula: desired = ceil(current * current_metric / desired_metric).

Metrics Server: HPA requires Metrics Server to collect real-time CPU and memory data from kubelets. Without it, HPA shows unknown targets and never scales.

Scale-Down Cooldown: By default, HPA waits 5 minutes before scaling down to avoid thrashing. This is configurable via --horizontal-pod-autoscaler-downscale-stabilization.

v2 API: HPA v2 supports multiple metrics including custom metrics (Prometheus queries via prometheus-adapter) and external metrics — not just CPU/memory.

SECTION 6 RBAC, Security Contexts & Service Accounts

Assignment 16 **RBAC — Role-Based Access Control**	Difficulty **Advanced**	Est. Time **45 min**

Objective

Create users and service accounts with specific permissions using Roles, ClusterRoles, RoleBindings, and ClusterRoleBindings.

Task

- Create a namespace called dev
- Create a ServiceAccount named dev-user in that namespace
- Create a Role allowing read-only access to Pods and Services in dev
- Bind the Role to the ServiceAccount
- Test permissions using kubectl auth can-i
- Create a ClusterRole for viewing nodes cluster-wide

Solution

```
# Create namespace
kubectl create namespace dev

# Create ServiceAccount
kubectl create serviceaccount dev-user -n dev

# role.yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: dev
rules:
- apiGroups: ['']
  resources: ['pods', 'pods/log', 'services', 'endpoints']
  verbs: ['get', 'list', 'watch']
- apiGroups: ['apps']
  resources: ['deployments', 'replicasets']
  verbs: ['get', 'list', 'watch']

# rolebinding.yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: dev-user-binding
  namespace: dev
subjects:
- kind: ServiceAccount
```

```
  name: dev-user
  namespace: dev
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io

kubectl apply -f role.yaml
kubectl apply -f rolebinding.yaml

# Test permissions
kubectl auth can-i list pods \
  --namespace=dev \
  --as=system:serviceaccount:dev:dev-user
# yes

kubectl auth can-i delete pods \
  --namespace=dev \
  --as=system:serviceaccount:dev:dev-user
# no

# ClusterRole for node access
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: node-reader
rules:
- apiGroups: ['']
  resources: ['nodes']
  verbs: ['get', 'list', 'watch']
```

Explanation

RBAC: Controls what actions users and service accounts can perform on Kubernetes resources. It is the primary access control mechanism in Kubernetes.

Role vs ClusterRole: Role is namespace-scoped (only works within one namespace). ClusterRole works across all namespaces AND includes cluster-level resources like nodes, PVs, and namespaces themselves.

RoleBinding vs ClusterRoleBinding: RoleBinding grants permissions within a namespace. ClusterRoleBinding grants cluster-wide permissions. You can bind a ClusterRole to a namespace with a RoleBinding — this restricts the ClusterRole to that namespace.

ServiceAccount: An identity for Pods (not humans). Each Pod runs as a ServiceAccount. The default ServiceAccount in each namespace has minimal permissions.

Security Best Practice

Apply Principle of Least Privilege: grant only the verbs and resources absolutely required. Avoid using cluster-admin ServiceAccounts in application Pods — this is a common security misconfiguration.

Assignment 17	Difficulty	Est. Time

Security Contexts — Run Containers Securely

Advanced | **35 min**

Objective

Configure security contexts at the Pod and container level to enforce the principle of least privilege for running containers.

Task

- Run a container as a non-root user
- Make the root filesystem read-only
- Drop all Linux capabilities and add back only what is needed
- Prevent privilege escalation
- Use a seccomp profile

Solution

```yaml
# secure-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: secure-pod
  annotations:
    seccomp.security.alpha.kubernetes.io/pod: runtime/default
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 1000
    runAsGroup: 3000
    fsGroup: 2000
    seccompProfile:
      type: RuntimeDefault
  containers:
  - name: app
    image: nginx:1.25-alpine
    securityContext:
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities:
        drop:
        - ALL
        add:
        - NET_BIND_SERVICE
    volumeMounts:
    - name: tmp
      mountPath: /tmp
    - name: var-run
      mountPath: /var/run
  volumes:
  - name: tmp
    emptyDir: {}
  - name: var-run
```

```
    emptyDir: {}

# Verify - check user inside container
kubectl exec secure-pod -- whoami   # should show uid=1000
kubectl exec secure-pod -- id
```

Explanation

runAsNonRoot: Prevents the container from running as root (UID 0). If the image defaults to root and no runAsUser is set, the Pod will fail to start.

readOnlyRootFilesystem: Makes the container's root filesystem immutable. Attackers who gain code execution cannot modify binaries or write persistence files. Use emptyDir volumes for directories the app needs to write to.

Capabilities: Linux capabilities are fine-grained root privileges. Drop ALL removes every capability, then add back only what is needed. NET_BIND_SERVICE allows binding to ports below 1024.

allowPrivilegeEscalation: Prevents the process from gaining more privileges than its parent. Always set to false — it blocks sudo, SUID binaries, and privilege escalation exploits.

SECTION 7 Advanced Workloads — Jobs, DaemonSets & CronJobs

Assignment 18

Jobs & CronJobs — Batch Processing

Difficulty	Est. Time
Intermediate	30 min

Objective

Create Kubernetes Jobs for one-time batch tasks and CronJobs for recurring scheduled work. Understand retry behavior.

Task

- Create a Job that runs a database migration script
- Make it run 3 completions in parallel
- Create a CronJob that runs a backup every day at 2 AM
- Manually trigger the CronJob
- Inspect Job logs and completion status

Solution

```
# job.yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: db-migration
spec:
  completions: 3
  parallelism: 2
  backoffLimit: 3
  activeDeadlineSeconds: 300
  ttlSecondsAfterFinished: 3600
  template:
    spec:
      restartPolicy: OnFailure
      containers:
      - name: migrator
        image: busybox
        command: ['sh', '-c', 'echo Running migration $JOB_COMPLETION_INDEX && sleep 5 &&
echo Done']
        env:
        - name: JOB_COMPLETION_INDEX
          valueFrom:
            fieldRef:
              fieldPath: metadata.annotations['batch.kubernetes.io/job-completion-index']

kubectl apply -f job.yaml
kubectl get jobs
kubectl get pods -l job-name=db-migration
kubectl logs -l job-name=db-migration
```

```yaml
# cronjob.yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: backup-job
spec:
  schedule: '0 2 * * *'
  concurrencyPolicy: Forbid
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 1
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          containers:
          - name: backup
            image: busybox
            command: ['sh', '-c', 'echo Backup started at $(date)']

# Manually trigger
kubectl create job --from=cronjob/backup-job manual-backup-$(date +%s)
```

Explanation

Job: Runs one or more Pods to completion. Unlike Deployments, Jobs are meant to finish. completions is how many successful completions needed. parallelism is how many Pods run simultaneously.

backoffLimit: How many times Kubernetes retries a failed Pod before marking the Job as failed. Uses exponential backoff between retries.

restartPolicy: Jobs must use Never or OnFailure. Never creates a new Pod on failure (good for debugging). OnFailure restarts the same Pod's container.

CronJob schedule: Uses standard cron syntax: minute hour day-of-month month day-of-week. '0 2 * * *' means 2:00 AM every day. concurrencyPolicy: Forbid prevents overlapping runs.

Cron Expression	Meaning
`0 2 * * *`	Every day at 2:00 AM
`*/5 * * * *`	Every 5 minutes
`0 0 * * 0`	Every Sunday at midnight
`0 9 1 * *`	1st of every month at 9 AM

Assignment 19	Difficulty	Est. Time
DaemonSets — Run on Every Node	**Intermediate**	**30 min**

Objective

Create a DaemonSet for a log collection agent that runs on every node. Understand node selectors to run on a subset of nodes.

Task

- Create a DaemonSet running Fluentd for log collection
- Mount the host's /var/log directory
- Add a node selector to run only on worker nodes
- Verify one Pod per node

Solution

```yaml
# daemonset.yaml
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: fluentd
  namespace: kube-system
  labels:
    app: fluentd
spec:
  selector:
    matchLabels:
      app: fluentd
  updateStrategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1
  template:
    metadata:
      labels:
        app: fluentd
    spec:
      tolerations:
      - key: node-role.kubernetes.io/control-plane
        effect: NoSchedule
      nodeSelector:
        kubernetes.io/os: linux
      serviceAccountName: fluentd
      containers:
      - name: fluentd
        image: fluent/fluentd-kubernetes-daemonset:v1-debian-elasticsearch
        env:
        - name: FLUENT_ELASTICSEARCH_HOST
          value: elasticsearch.logging.svc.cluster.local
        resources:
          requests:
            cpu: 100m
            memory: 200Mi
          limits:
            cpu: 500m
            memory: 500Mi
        volumeMounts:
        - name: varlog
          mountPath: /var/log
```

```
          readOnly: true
        - name: varlibdockercontainers
          mountPath: /var/lib/docker/containers
          readOnly: true
      volumes:
      - name: varlog
        hostPath:
          path: /var/log
      - name: varlibdockercontainers
        hostPath:
          path: /var/lib/docker/containers

kubectl apply -f daemonset.yaml
kubectl get pods -n kube-system -l app=fluentd -o wide
# One Pod per node
```

Explanation

DaemonSet: Ensures exactly one Pod runs on each node (or a selected subset). New nodes joining the cluster automatically get the DaemonSet Pod.

Use Cases: Log collectors (Fluentd, Logstash), monitoring agents (Prometheus Node Exporter, Datadog Agent), network plugins (CNI agents), storage drivers, and security scanners.

hostPath Volume: Mounts a directory from the host node's filesystem. Used by monitoring and log agents to access host files. Be careful: this gives the container access to host files.

Control Plane Toleration: Control plane nodes are tainted by default. Add the toleration if you want your DaemonSet to also run on control plane nodes (like a security scanner that needs all nodes).

SECTION 8 Observability — Logging, Debugging & kubectl Mastery

Assignment 20 **Debugging Pods — Systematic Troubleshooting**	Difficulty **Intermediate**	Est. Time **35 min**

Objective

Debug five common Kubernetes failure scenarios systematically using kubectl commands.

Scenarios & Solutions

Scenario 1: Pod in Pending State

```
kubectl get pod stuck-pod
# STATUS: Pending

kubectl describe pod stuck-pod
# Look at Events section:
# 0/3 nodes are available: insufficient cpu
# -> Fix: Reduce resource requests or add nodes

# OR: no nodes matched node selector
# -> Fix: Check node labels match pod's nodeSelector
```

Scenario 2: CrashLoopBackOff

```
kubectl get pods
# STATUS: CrashLoopBackOff

# Check logs
kubectl logs crash-pod
kubectl logs crash-pod --previous   # logs from crashed container

# Check events
kubectl describe pod crash-pod | grep -A 20 Events
# Common causes: app config error, missing env vars, file not found
```

Scenario 3: ImagePullBackOff

```
kubectl describe pod bad-image-pod | grep -A5 Events
# Failed to pull image 'nginx:nosuchtag': rpc error
# -> Fix: correct the image tag
# -> Fix for private registry: create imagePullSecret

kubectl create secret docker-registry regcred \
  --docker-server=ghcr.io \
  --docker-username=myuser \
  --docker-password=mytoken

# Add to Pod spec
spec:
  imagePullSecrets:
```

```
  - name: regcred
```

Scenario 4: Service Not Routing to Pods

```
# Service exists but curl returns nothing
kubectl get svc my-svc
kubectl get endpoints my-svc
# Endpoints: <none>  <- label mismatch!

# Compare service selector with pod labels
kubectl get svc my-svc -o yaml | grep selector -A5
kubectl get pods --show-labels
# Fix: ensure pod labels match service selector
```

Scenario 5: Pod Evicted

```
kubectl get pods
# STATUS: Evicted

kubectl describe pod evicted-pod
# The node was low on resource: memory  (eviction)
# -> Fix: set resource limits, check node memory
kubectl top nodes
kubectl top pods
```

kubectl Debugging Toolkit

Command	What It Tells You
`kubectl describe pod <name>`	Full lifecycle, events, resource assignments
`kubectl logs <name> --previous`	Logs from the last crashed container
`kubectl exec -it <name> -- sh`	Interactive shell for live debugging
`kubectl get events --sort-by='.lastTimestamp'`	Cluster-wide events newest first
`kubectl top pods`	Current CPU/memory usage (needs Metrics Server)
`kubectl debug node/<node>`	Root shell on node for kernel-level debugging
`kubectl get pod -o wide`	Which node each Pod is on
`kubectl port-forward pod/<name> 8080:80`	Forward local port to Pod for direct testing

Assignment 21 **kubectl JSONPath & Custom Output**	Difficulty **Intermediate**	Est. Time **25 min**

Objective

Master kubectl output formatting with JSONPath and custom columns to extract exactly the data you need.

Solutions

```
# Get all pod IPs
kubectl get pods -o jsonpath='{.items[*].status.podIP}'

# Get pod names and their node
kubectl get pods -o jsonpath='{range
.items[*]}{.metadata.name}{"\t"}{.spec.nodeName}{"\n"}{end}'

# Custom columns output
kubectl get pods -o custom-columns=\
  NAME:.metadata.name,\
  STATUS:.status.phase,\
  NODE:.spec.nodeName,\
  IP:.status.podIP

# Get all images running in the cluster
kubectl get pods -A -o jsonpath='{range .items[*]}{.spec.containers[*].image}{"\n"}{end}'
| sort | uniq

# Get Secrets names in all namespaces
kubectl get secrets -A -o custom-columns=NS:.metadata.namespace,NAME:.metadata.name

# Filter pods by label, get just names
kubectl get pods -l app=nginx -o jsonpath='{.items[*].metadata.name}'

# Get Deployment image
kubectl get deployment web-app \
  -o jsonpath='{.spec.template.spec.containers[0].image}'
```

Explanation

JSONPath is a query language for JSON. kubectl -o jsonpath lets you navigate the Kubernetes API object tree and extract specific fields. The range keyword iterates over arrays. Custom columns give you table output with exactly the columns you want — perfect for scripts and dashboards.

SECTION 9 Helm — Package Management for Kubernetes

Assignment 22

Helm Basics — Install, Upgrade & Rollback

Difficulty	Est. Time
Intermediate	40 min

Objective

Install Helm, add repos, deploy an application chart, upgrade it with custom values, and roll back a failed release.

Solution

```
# Install Helm
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
helm version

# Add popular repos
helm repo add stable https://charts.helm.sh/stable
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update

# Search for charts
helm search repo nginx
helm search hub wordpress

# Install nginx with default values
helm install my-nginx bitnami/nginx \
  --namespace web \
  --create-namespace

helm list -n web
helm status my-nginx -n web

# Custom values override
# values-override.yaml
replicaCount: 3
service:
  type: LoadBalancer
resources:
  requests:
    cpu: 100m
    memory: 128Mi

helm upgrade my-nginx bitnami/nginx \
  -n web \
  -f values-override.yaml

helm history my-nginx -n web
```

```
# Rollback to revision 1
helm rollback my-nginx 1 -n web

# Uninstall
helm uninstall my-nginx -n web
```

Explanation

Helm Chart: A package of pre-configured Kubernetes resources. A single chart can deploy a complex multi-component application (Deployment, Service, Ingress, ConfigMap, PVC) in one command.

Release: Each installation of a chart is a release with a name. You can install the same chart multiple times in different namespaces with different names and different values.

Values: Charts are parameterized. The default values are in values.yaml. You override specific values with -f or --set. This makes charts reusable across dev, staging, and production.

Rollback: Helm keeps a history of releases. helm rollback is faster and safer than manually reverting YAML — it reinstates the exact previous set of manifests.

Assignment 23 **Create Your Own Helm Chart**	Difficulty **Advanced**	Est. Time **50 min**

Objective

Build a Helm chart from scratch for the DevOps Shack web app. Include Deployment, Service, Ingress, and configurable values.

Solution

```
# Scaffold the chart
helm create devopsshack-app
tree devopsshack-app

# Chart.yaml
apiVersion: v2
name: devopsshack-app
description: DevOps Shack Web Application
version: 1.0.0
appVersion: '2.0'

# values.yaml
replicaCount: 2
image:
  repository: nginx
  tag: '1.25'
  pullPolicy: IfNotPresent
service:
  type: ClusterIP
  port: 80
ingress:
  enabled: true
  host: devopsshack.example.com
```

```
  ingressClassName: nginx
resources:
  requests:
    cpu: 100m
    memory: 128Mi
  limits:
    cpu: 200m
    memory: 256Mi
config:
  appEnv: production
  logLevel: info

# templates/deployment.yaml (key part)
spec:
  replicas: {{ .Values.replicaCount }}
  template:
    spec:
      containers:
      - name: {{ .Chart.Name }}
        image: {{ .Values.image.repository }}:{{ .Values.image.tag }}
        imagePullPolicy: {{ .Values.image.pullPolicy }}
        resources:
          {{- toYaml .Values.resources | nindent 10 }}

# Lint and validate
helm lint devopsshack-app
helm template devopsshack-app devopsshack-app/ --debug

# Install
helm install devopsshack devopsshack-app/

# Override for production
helm install devopsshack-prod devopsshack-app/ \
  --set replicaCount=5 \
  --set ingress.host=prod.devopsshack.com
```

Explanation

Go Templating: Helm uses Go's template language. {{ .Values.key }} inserts values. {{- }} trims whitespace. toYaml | nindent renders a YAML block with proper indentation.

_helpers.tpl: Contains reusable template functions. The boilerplate chart creates helpers for the app name and labels so they are consistent across all templates.

helm template: Renders the templates locally without installing. Use this to verify the generated YAML before applying to the cluster.

SECTION 10 — Production Patterns — PodDisruptionBudgets, Init Containers & More

Assignment 24	Difficulty	Est. Time
Init Containers — Pre-flight Checks	Intermediate	30 min

Objective

Use Init Containers to perform prerequisite tasks before the main application starts. Classic use case: wait for a database.

Solution

```
# init-container.yaml
apiVersion: v1
kind: Pod
metadata:
  name: app-with-init
spec:
  initContainers:
  - name: wait-for-db
    image: busybox
    command: ['sh', '-c',
      'until nc -z postgres-svc 5432; do
        echo waiting for database;
        sleep 2;
      done;
      echo Database is ready']
  - name: run-migrations
    image: myapp-migrator:latest
    command: ['python', 'manage.py', 'migrate']
    env:
    - name: DB_URL
      valueFrom:
        secretKeyRef:
          name: db-credentials
          key: DATABASE_URL
  containers:
  - name: app
    image: myapp:latest
    ports:
    - containerPort: 8000

kubectl apply -f init-container.yaml
kubectl get pods   # STATUS: Init:0/2, Init:1/2, Running
kubectl logs app-with-init -c wait-for-db
kubectl logs app-with-init -c run-migrations
```

Explanation

Init Containers: Run to completion sequentially BEFORE main containers start. If any init container fails, Kubernetes restarts it (per the Pod restartPolicy). Main containers don't start until all init containers succeed.

Use Cases: Wait for dependencies (databases, queues, APIs), run database migrations, fetch secrets or configs, clone git repos into shared volumes, register with service mesh.

Separate Image: Init containers can use different images from main containers. A migrator init container uses your app image with migration tools, while the main container is a slim runtime image.

Assignment 25 **PodDisruptionBudgets — Ensure High Availability During Updates**	Difficulty **Advanced**	Est. Time **30 min**

Objective

Create a PodDisruptionBudget to prevent too many Pods from being terminated simultaneously during node maintenance or cluster upgrades.

Solution

```
# pdb.yaml
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: web-app-pdb
spec:
  minAvailable: 2
  # OR use maxUnavailable:
  # maxUnavailable: 1
  selector:
    matchLabels:
      app: web-app

kubectl apply -f pdb.yaml
kubectl get pdb web-app-pdb
# ALLOWED DISRUPTIONS shows how many can be terminated

# Simulate node drain (triggers PDB)
kubectl drain <node-name> --ignore-daemonsets --delete-emptydir-data
# Kubernetes ensures minAvailable=2 Pods stay up during drain

# Uncordon the node after maintenance
kubectl uncordon <node-name>
```

Explanation

PodDisruptionBudget: Limits the number of Pods that can be voluntarily disrupted simultaneously. Protects your app during node drains, cluster upgrades, and scaling events.

minAvailable vs maxUnavailable: minAvailable=2 means at least 2 Pods must be running before any disruption is allowed. maxUnavailable=1 means at most 1 Pod can be unavailable at a time.

Voluntary vs Involuntary: PDB only controls voluntary disruptions (kubectl drain, node upgrades). Involuntary disruptions (hardware failure, out of memory) are not controlled by PDB.

Production Rule

Always create a PDB for any Deployment with more than 1 replica that is customer-facing. Without a PDB, a node drain could take down all Pods if they all happen to be on the same node.

Quick Reference — Essential kubectl Commands

Action	Command
Get all resources	kubectl get all -n <namespace>
Watch pods live	kubectl get pods -w
Follow logs	kubectl logs -f <pod> -c <container>
All namespaces	kubectl get pods -A
Describe resource	kubectl describe <type> <name>
Apply manifest	kubectl apply -f <file.yaml>
Delete resource	kubectl delete -f <file.yaml>
Force delete pod	kubectl delete pod <name> --force --grace-period=0
Edit resource live	kubectl edit deployment <name>
Scale deployment	kubectl scale deployment <name> --replicas=5
Rollout status	kubectl rollout status deployment/<name>
Rollback	kubectl rollout undo deployment/<name>
Port forward	kubectl port-forward svc/<name> 8080:80
Copy files	kubectl cp <pod>:/path /local/path
Resource usage	kubectl top pods / kubectl top nodes
API resources	kubectl api-resources
Explain field	kubectl explain deployment.spec.strategy
Auth check	kubectl auth can-i create pods --as=<user>

Kubernetes Objects Cheat Sheet

Object	Purpose
Pod	Smallest deployable unit. Wraps containers.
Deployment	Manages ReplicaSets. Self-healing, rolling updates.
ReplicaSet	Ensures N pod replicas. Managed by Deployment.
StatefulSet	Stable identity pods. For databases and stateful apps.
DaemonSet	One pod per node. For agents and monitors.
Job	Run pods to completion. For batch tasks.

`CronJob`	Scheduled Jobs. For recurring tasks.
`Service`	Stable network endpoint. Load balances to pods.
`Ingress`	HTTP/HTTPS routing. TLS termination.
`ConfigMap`	Non-sensitive config as key-value or files.
`Secret`	Sensitive data (base64). Passwords, tokens, certs.
`PersistentVolume`	Storage provisioned in the cluster.
`PersistentVolumeClaim`	User request for storage.
`StorageClass`	Dynamic volume provisioning profile.
`Namespace`	Virtual cluster for isolation and RBAC scoping.
`ServiceAccount`	Identity for Pods to call Kubernetes API.
`Role / ClusterRole`	Define permissions on resources.
`RoleBinding`	Grant Role to user/group/serviceaccount.
`NetworkPolicy`	Firewall rules for pod-to-pod traffic.
`HorizontalPodAutoscaler`	Auto-scale pods based on metrics.
`PodDisruptionBudget`	Limit voluntary pod disruptions.
`LimitRange`	Default resource limits per namespace.
`ResourceQuota`	Total resource cap per namespace.

You've completed

Learn Kubernetes By Doing

Continue your DevOps journey at:

devopsshack.com

YouTube: DevOps Shack | Instagram: @devopsshack | LinkedIn: Aditya Jaiswal